Design And Analysis for Institute Time Table Schedular

*CS378 : Design and Analysis of Algorithm

Muhammad Bin Waseem
dept. of Cyber Security
reg no : 2023403

Muhammad Shaheem
dept. of Cyber Security
reg no : 2023507

Muhammad Shaheer
dept. of Cyber Security
reg no : 2023509

Muhammad Zaeem Nawaz
dept. of Cyber Security
reg no : 2023550

I. Problem Statement

Timetable planning in academic institutions involves a complex combinatorial optimization task that often results in conflicts when done manually. The common problems faced are the assignment of a single faculty member to multiple classes in a single period, assignment of a single lecture room to multiple periods, and ineffective scheduling of classes. Manual timetable planning also causes an imbalance in lecture scheduling, resulting in long stretches of consecutive lectures or blank periods in students' schedules.

These problems lead to reduced academic performance and ad- ditional costs. Moreover, manual generation of timetables is both time-consuming and prone to errors, making it unsuitable for in- stitutional settings such as GIK Institute. Hence, there is a need for an automated timetable planning system that produces valid timetables while considering institutional constraints.

II. Assumptions

The design of the algorithm is made under a number of simplifications for making the solution feasible. It is assumed that the total number of participants in any two courses cannot exceed the number of seats in the allocated lecture hall. The first digit in the code of any course represents the year of its delivery. The time table includes eight time periods per day, with each period being 50 minutes long. These periods remain unchanged within the framework of scheduling.

The priority for placing classes in the main lecture hall is given to the first-year courses due to their high enrollment strength.

It is also assumed that input data will be formatted in the correct form and will contain no missing or null values. Moreover, classes are scheduled only on the first five days of the week.

There are also a number of organizational constraints. It is assumed that all courses starting with the prefix "PH" are delivered in the FES faculty, while others are distributed according to departmental specifications. Civil Engineering students should be preferred for the ACB buildings. Sections taught by the same professor can be combined if possible. Some laboratory classes, like PH101, MM101 can also be combined under if slots are unavailable.

III. Constraint Handling Strategy

Constraint Handling is comprised of two types, hard constraints and soft constraints, all of which are incorpo-rated in the scheduling process.

A. Hard Constraints

Hard constraints are strictly implemented through bit-masks for conflict detection. All teachers, classrooms, and student sections have a representation of their occupancy at any point in time that provides constant time conflict detection. No course can be scheduled into a slot if there exists a conflict among these three parameters.

Furthermore, the following policies are implemented:

- Courses prefixed with `PH` are restricted to the Faculty of Engineering Sciences (FES), and the same rule is true for other departments.
- Sections for civil engineering are preferred to be scheduled in the ACB building.
- Laboratory classes such as PH101, IF101, and MM101 can be combined provided certain conditions.
- Another constraint based on a ratio ensures that scheduling past a certain density level would constitute permanent conflict.

B. Soft Constraints

Soft constraints, on the other hand, are dealt with through a scoring function, which penalizes bad scheduling behavior. First year students are favored during the allocation of lecture slots in large lecture halls due to strong enroll- ment strength. Moreover, the system minimizes overlapping consecutive lectures and unnecessary gaps between them. Friday Free Mode allows fewer Friday classes in the sche- dule. Once this mode is activated, a large penalty is assigned for each Friday time slot to the cost function. Consequently, such conflicting schedules are automatically excluded from being generated. The schedule that contains maximum sections with zero Friday classes will be produced without violating any other constraints.

C. Conflict Resolution

Unresolved conflicts are handled using a Min-Conflicts strategy. Courses involved in conflicts are iteratively reassigned while temporarily allowing constraint violations. A

probabilistic acceptance mechanism inspired by simulated annealing further enhances solution quality by avoiding premature convergence.

IV. Input and Output Specification

The system requires structured inputs in Excel file formats (XLSX). The record has fields like serial number, course code, credit hours, identifiers for instructor and sections and a single-digit integer value indicating the year of study (from 1 to 4). All the fields must be present and have values which are not null. Room numbers and buildings are also included in the system. Here rooms are categorized as Lecture halls or Laboratories. If no building information is available, a default setting is assumed.

The system provides three types of outputs: Section-wise Timetable, Building-wise Timetable, and Instructor Time Table. The section-wise timetable contains schedules created for each student section individually. The building-wise timetable contains schedules shows the time table for a complete buildings rooms the class timings in those rooms. The instructor time table shows the lectures of each individual instructor along with the time slot allotted. Both Excel and PDF formats can be used to export the output. There is also an interactive dashboard through which the timetable can be viewed in real-time.

V. Algorithm Design and Implementation

A. Overview

The proposed algorithm uses a mixed greedy con- structive heuristic combined with the Min-Conflicts algorithm supplemented by Simulated Annealing. The task is represented as a constraint optimization problem. Its goal is to assign each course a time slot, room, instructor and class section in such a way that all hard constraints are satisfied, andsoft constraints are optimized.

Because of the NP-hard nature of the timetable scheduling task, finding optimal solution is computationally inefficient. Thus, heuristic methods are utilized to find approximations of feasible solutions in time.

B. Data Representation

Time is encoded in 40-bit bitmask, consisting of five days, each having eight possible time slots. Each entity maintains its own bitmask, which denotes occupied slots. Teacher, room and section schedules are kept apart to enable quick conflict search using bit manipulation.

C. Greedy Construction Phase

During the initial step, all courses are ranked according to a priority heuristic that gives preference to laboratories,lower semester courses, courses with less possible rooms and those with more credit hours. Such approach guarantees that the most constrained courses will be scheduled earlier.

Each course is assigned to the best fitting room and slot taking into account the number of free spots and the preferred building and number of conflicting courses. Otherwise, it is transferred to a separate unassigned queue.

D. Refinement Phase

The second stage employs the Min-Conflicts heuristic algorithm that resolves all existing conflicts using iteration procedure. During each iteration, a sample of courses that violate constraints is generated, and the most problematic one is picked out. Then, its time slot is reassigned to the best position.

To avoid local minima, Simulated Annealing is applied using a temperature parameter that gradually decreases over time. This allows occasional acceptance of suboptimal solutions during early iterations, improving exploration.

VI. Time Complexity Analysis

Let C represent the number of courses, R the number of rooms, T the number of instructors, S the number of student sections, K the number of refinement iterations, and M the number of restarts.

A. Greedy Algorithm

The time complexity of greedy phase is $O(C \cdot R \cdot (T+S))$. For each of the C courses, the algorithm iterates through every R available rooms. While checking each room's feasibility, the algorithm verify instructor and section availability over a fixed schedule of 40 slots. As the number of time slots is a constant,it does not contribute to the time complexity therefore the dominant term is $O(C \cdot R \cdot (T + S))$.

B. MinConflict Algorithm

In refinement phase the time complexity is $O(K \cdot R \cdot (T+S))$. This phase executes for K iterations, and in each iteration the algorithm *FindBestAssignment*, is called which checks all rooms and computes feasibility using instructor and section availability checks. While doing so, conflict resolution involves instructor and section availability checking and adds additional time $O(C)$ to the process, Yet, in practice, in university-level instances of timetable scheduling, the term $R \cdot (T + S)$ dominates or equals the term C, leading to the time complexity being C, allowing the refinement phase to be expressed as $O(K \cdot R \cdot (T + S))$.

The overall algorithm is executed within M independent restarts. Therefore, combining both phases yields the final worst-case complexity:

$$O(M \cdot (C + K) \cdot R \cdot (T + S)) \tag{1}$$

TABLE I
Time Complexity Breakdown

Component	Complexity
Course Sorting	$O(C \log C)$
Phase 1: Greedy Construction	$O(C \cdot R \cdot (T + S))$
Phase 2: Min-Conflicts (per restart)	$O(K \cdot R \cdot (T + S))$
Overall Best Case	$\mathbf{O(C \cdot R \cdot (T + S))}$
Overall Average Case	$\mathbf{O(M \cdot (C + K) \cdot R \cdot (T + S))}$
Overall Worst-Case	$\mathbf{O(M \cdot (C + K) \cdot R \cdot (T + S))}$

VII. Space Complexity Analysis

The space complexity of the implemented scheduling system depends on the retained data structures for encoding the state of the schedule, constraints, and assignments. This algorithm is further optimized using bit masking and grid representations, thus minimizing memory requirements without degrading conflict detection performance.

A. Valid Room Storage

For every course C, there will be a precomputed set of valid rooms taking into account constraints like building preference and lab suitability. In the worst case scenario when no filtering can be done, the course can be mapped to all the available rooms R. This leads to a space complexity of

$$O(C \cdot R) \tag{2}$$

B. Scheduling Grids

The algorithm uses three schedules grids to facilitate constant-time conflict checking and allocation of teachers' schedule grids, sections' schedule grids, and rooms' schedule grids. Each schedule grid consists of 40 time slots for each entry. Due to the constant number of time slots involved, the algorithm runs with:

$$O(T + S + R) \tag{3}$$

C. Busy Bitmask Storage

Each instructor, class, and classroom keeps a 40-bit integer bitmask, which facilitates quick checks for conflicts through bitwise manipulation.The memory requirement is:

$$O(T + S + R) \tag{4}$$

D. Conflict Tracking Overhead

During the Min-Conflicts refinement phase, extra temporary storage is used for the `unassigned_courses` list and per-course conflict counters. This gives us :

$$O(C) \tag{5}$$

TABLE II
Space Complexity Breakdown

Component	**Complexity**
Valid Room Storage	$O(C \cdot R)$
Scheduling Grids (Teacher, Section, Room)	$O(T + S + R)$
Busy Bitmasks	$O(T + S + R)$
Conflict Tracking	$O(C)$
Overall Space Complexity	$\mathbf{O(C \cdot R + T + S)}$

VIII. Results

The building based time table displays a grid of rooms/time slots and what course is taking place there accurately.

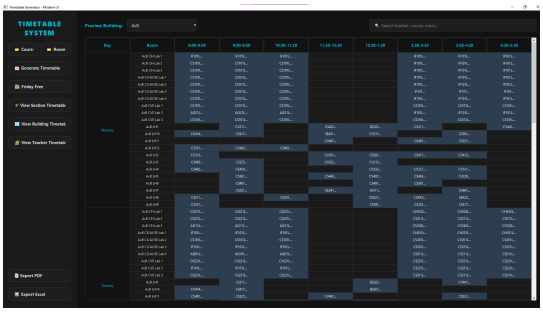

Fig. 1. Building Time Table

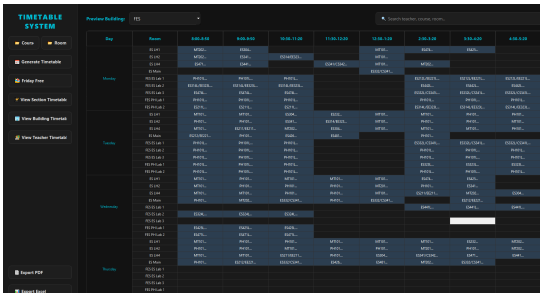

Fig. 2. Building Time Table 2

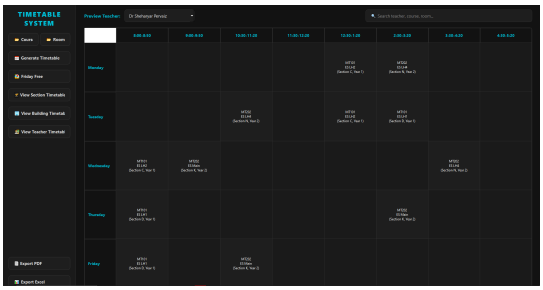

Fig. 3. Instructor Based Time Table

figure 3 Displays the schedule for the instructor where and when does the instructor have classes.

Figure 4 shows the time table the specific section of specific year and ensures that there are no unnecessary gaps or too many consecutive classes.

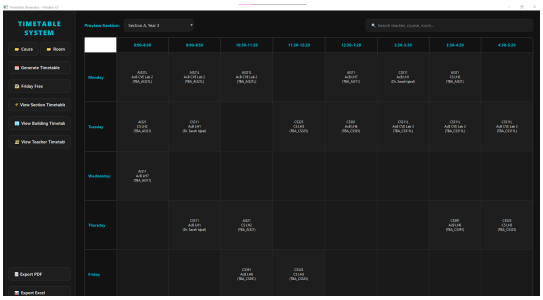

Fig. 4. Section Based Time Table

IX. Conclusion

The suggested hybrid algorithm is able to generate viable schedules by utilizing the greedy approach for initializing the schedules and resolving conflicts using heuristic approaches. Bitmask representation is used in the process to achieve O(1) conflict checking, as well as convergence and solution improve- ment.he system is scalable, efficient, and suitable for real-world academic scheduling environments.

References

[1] A. A. Wano, *Introduction to the Design and Analysis of Algorithms*. Pearson Education, 2019.

[2] M. W. Carter and G. Laporte, "Recent developments in practical course timetabling," in *Proc. 2nd Int. Conf. Practice and Theory of Automated Timetabling (PATAT)*, Toronto, Canada, 1997, pp. 3–19.

[3] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[4] R. Lewis, *A Guide to Graph Colouring: Algorithms and Applications*. Springer, 2016.

[5] K. Z. Zamli, F. Din, and S. Baharom, "A harmony search algorithm for university course timetabling," in *Proc. Int. Conf. Intelligent Systems Design and Applications*, 2007, pp. 120–125.

Appendix

Appendix: Algorithm Pseudocode

Algorithm 1 Automated Timetable Scheduler

procedure SOLVE(C, R, K, M)
  PREPROCESS(C, R) ▷ Filter valid rooms per course by type & building
  SORT C by (*is_lab* ↓, *semester* ↑, $|valid\_rooms|$ ↑, *credit_hours* ↓)
  for $restart \leftarrow 1$ **to** M **do**
    RESET all bitmasks, grids, $conflict\_count$
    if $restart > 1$ **then**
      SHUFFLE $valid\_rooms$ for each $c \in C$; RE-SORT C
    end if
    $U \leftarrow \emptyset$ ▷ Unassigned course set
    for each $c \in C$ **do** ▷ Phase 1: Greedy Construction
      $(r, m) \leftarrow$ FIND_BEST($c,\ allow_col = $ False)
      if $r \neq$ NIL **then** ASSIGN(c, r, m)
      else $U \leftarrow U \cup \{c\}$
      end if
    end for
    if $U = \emptyset$ **then return** *Success*
    end if
    for $i \leftarrow 1$ **to** K **do** ▷ Phase 2: Min-Conflicts + Simulated Annealing
      if $U = \emptyset$ **then return** *Success*
      end if
      $T \leftarrow 50.0 \times (1 - i/K)$ ▷ Decaying SA temperature
      $c \leftarrow \arg\max_{x \in \text{SAMPLE}(U,\ 5)}\ x.conflict\_count$
      $(r, m, \Delta) \leftarrow$ FIND_BEST($c,\ allow_col = $ True, T)
      for each $d \in \Delta$ **do**
        $d.conflict\_count \leftarrow d.conflict\_count + 1$
        UNASSIGN(d); $U \leftarrow U \cup \{d\}$
      end for
      ASSIGN(c, r, m); $U \leftarrow U \setminus \{c\}$
    end for
  end forreturn *Failure*
end procedure

Algorithm 2 Find Best Assignment

function FIND_BEST($c,\ allow\_col,\ T$)
  $best\_score \leftarrow \infty$; $best\_r \leftarrow$ NIL; $best\_m \leftarrow 0$
  for each $r \in c.valid\_rooms$ **do**
    if $c.is\_lab$ **then**
      for each consecutive slot pattern $p \in$ LAB_MASKS **do**
        $\Delta \leftarrow$ GET_COLLIDERS(c, r, p)
        if not $allow\_col$ **and** $\Delta \neq \emptyset$ **then continue**
        end if
        $score \leftarrow$ COST(Δ, r, c, p, T)
      end for
    else ▷ Lecture course
      for each slot $s \in \{0, \ldots, 39\}$ **do**
        $\Delta_s \leftarrow$ GET_COLLIDERS(c, r, s)
        if not $allow\_col$ **and** $\Delta_s \neq \emptyset$ **then continue**
        end if
        $score_s \leftarrow$ SLOT_COST(Δ_s, s, c, T)
      end for
      Greedily select $c.credit\_hours$ lowest-score slots
      Apply same-day penalty $+800$ to remaining slots on each chosen day
      $score \leftarrow \sum score_s +$ ROOM_COST(r, c)
    end if
    if $score < best\_score$ **then**
      $best\_score \leftarrow score$; $best\_r \leftarrow r$; $best\_m \leftarrow m$
    end if
  end forreturn ($best\_r$, $best\_m$, colliders of best assignment)
end function

Algorithm 3 Assign and Unassign

procedure ASSIGN(c, r, m)
  $teacher\_busy[t]\ |= m \quad \forall\, t \in c.instructor\_ids$
  $section\_busy[s]\ |= m \quad \forall\, s \in c.section\_ids$
  $room\_busy[r]\ |= m$
  for each slot i where bit i of $m = 1$ **do**
    $teacher\_grid[t][i] \leftarrow c \quad \forall t$; $room\_grid[r][i] \leftarrow c$; $section\_grid[s][i] \leftarrow c \quad \forall s$
  end for
end procedure
UNASSIGN is identical: replace $|= m$ *with* $\&=\sim m$ *and set grid entries to NIL.*